```python
#Importing the required libraries
import Data #Data on school graph to be enacted upon by algorithm
from functools import reduce #higher order function to shorten what would be much longer for loops into one line (line 57)
#I wont explain the purpose of any of the below as they already explained in the Data.py file so go there for explanation
CorridorContent = Data.CorridorContent
CorridorDists = Data.CorridorDists
NodeConnections = Data.NodeConnections
NodeCoords = Data.NodeCoords
CorridorFloorMapper = Data.CorridorFloorMapper

def extract_minval(list): #Function to extract the minimum value from a list (used on line 27)
    min_val_index = 0 #initialise the mininumu value index variable
    for current_node in range(1, len(list)): #loop through every item in the list
        if list[current_node][0] < list[min_index][0]: #If the distance for term i in queue is less than the current minimum distance to term min index
            min_index = current_node #set new min_index to current_node
    return list.pop(min_val_index) #return the item of lowest value form the list

def dijkstra_recursive(graph, start, end, visited=None, distances=None, queue=None, predecessors=None):
    if visited is None: #At the start when first running such this runs once
        visited = set() #created the visited set
        distances = {node: float('inf') for node in graph} #create dicitonary of nodes with each distance to the staart node as infinity
        distances[start] = 0 #set distance from start node to start node as 0 (obvisouly)
        queue = [(0, start)]  # (distance, node) #the start node is added to the queue
        predecessors = {} #predecessor list to track the path being taken
    if not queue: #is the queue is empty so done
        return distances, reconstruct_path(predecessors, start, end)  # Return distances and path

    current_distance, current_node = extract_minval(queue) #take node from queue with the current lowest distance to the node were currently at at set this to be the new current node were currently looking
at


    if current_node in visited: #if current node is already visited
        return dijkstra_recursive(graph, start, end, visited, distances, queue, predecessors)#continue recrusivly to the next node (as no need to do the below checks then)

    visited.add(current_node) #mark current node as visited

    for neighbor, weight in graph[current_node].items(): #convert dictionary into a list of key value pairs using .items and loop through the neighbour nodes to the current node and its weighting
        if neighbor not in visited: #if nodes isnt visited
            new_distance = current_distance + weight #calculate a new distance to that node (from start node)
            if new_distance < distances[neighbor]: #if the new distances is lower than the current lowest distance to the neighbour (from start node)
                distances[neighbor] = new_distance #set new distance to be the lowest distance to neighbour (from start nodw)
                predecessors[neighbor] = current_node #set the current node as the closest node to the neighbour node
                queue.append((new_distance, neighbor))#add the new distance and neighbour to the queue

    return dijkstra_recursive(graph, start, end, visited, distances, queue, predecessors)#continue recrusivly to the next node

def reconstruct_path(predecessors, start, end): #function to obtain the path of the result from dijkstras algorithm
                                                #predecessors contains the shortest link between nodes on the path
                                                #appears on line 26
    path = [] #initialise the path
    current = end #start from the end node
    while current in predecessors: #while current node is in predecessors
        path.append(current) #add current node to the path
        current = predecessors[current] #set current node to the shortest next neighbouring node on the path
    path.append(start) #once current node is no long ein predecessors then the start node has been reached, so add start node to path
    return path[::-1]  # Reverse the path as path was obtained in reverse order


def total_path_dist(distances,path):#function to return the total distance of the path, appears on line 117
    return reduce(lambda x,y: x+y,[distances[corridor] for corridor in path]) #go through each item in the list and add them together, returning the result
                                                                             #list looks through each corridor in the path and adds the distance of it to the list

def stripper(currentcorridor,currentroom,mapper): #removes the unecessary information from rooms to be more easily referenced (cleaning data)
                                                #mapper is corridor contents turned into a list under .items(),
                                                # in such format [Corridor1,[room1,room2,...]],[Corridor2,[room1,room2,...]],... etc.
                                                # appears on line 91
    char =  mapper[currentcorridor][1][currentroom] #takes the room from list in questions
    char_ = "" #intialise the character variable
    for x in char: #for each character in the room
```

```
        if x != ",": #if the character isnt a comma
            char_ += x #add the character to the character variable
        elif x == ",": #if the characteer is comma
            break #break or stop the for loop there
    return char_ #hence returns the room up until a comma is reached

def linkcorridortonode(corridor): #self explanatory but returns the link between each corridor and its two connection nodes (as each corridor has a node either side)
                                  #appears on line 92
    nodes = [] #initialise the nodes list
    for node in NodeConnections: #for each node that exists
        for neighbournode in NodeConnections[node]: #for each neighbouring node to that current node
            if NodeConnections[node][neighbournode] == corridor: #if the link (corridor) between these two nodes is the corridor of interest
                nodes.append(node) #add the current node to the nodes list
                nodes.append(neighbournode) #add the neighbouring node to the nodes list
        if len(nodes) == 2: #if the length of the nodes list is 2, then as any corridor can only have 2 node connections, then we know we are done
            break #so stop the for loop
    return nodes #return the nodes list

def findroomtocorridor(roomwhere,mapper):#function to find what corridor a room lies within
    currentcorridor = 0 #initialise the current corridor index
    currentroom = 0 #initialise the current room index
    while True: #and indefinte while loop, but will break at some point no matter what through a return
        try:
            roomsearch=stripper(currentcorridor,currentroom,mapper)[1:] #using the stripper function, set the current room up
            if roomwhere == roomsearch: #if the current room is the room of interest
                return [roomwhere,mapper[currentcorridor][0],linkcorridortonode(mapper[currentcorridor][0]),mapper[currentcorridor][1][currentroom],findfloor(mapper[currentcorridor][0])]
            # return the current room (which is the room of interest)
            # the corridor that the room lies in,
            # the nodes this corridor is connected to,
            # the room with the prefixing spacing id (explained in Data.py under CorridorContent)
            # the floor that this corridor lies on
            elif (currentcorridor == len(mapper)-1)and(currentroom==len(mapper[currentcorridor][1])-1): #if the current corridor index is the last corridor in list
                                                                                                        # and the current room index is equal to the number of rooms of the last corridor
                print("room not in map") #the room cannot be in the map
                return None #return nothing
            elif (len(mapper[currentcorridor][1])==0)or(currentroom == len(mapper[currentcorridor][1])-1): #if the number of rooms in a corridor is 0,
                                                                                                           #or the current room index is the final room in the corridor
                currentcorridor += 1 #increment the corridor index by one
                currentroom = 0 #reset the current room index
            else:
                currentroom += 1 #otherwise increment the current room index
        except IndexError: #should an index error occur then repeat the code from lines 95-102
            if (currentcorridor == len(mapper)-1)and(currentroom==len(mapper[currentcorridor][1])-1):
                print("room not in map")
                return None
            elif (currentroom == len(mapper[currentcorridor][1])-1)or(len(mapper[currentcorridor][1])==0):
                currentcorridor += 1
                currentroom = 0

def min_dist(info,graph): #function to obtain the mininum distance path from a start node to end node
                          #info is a list containing all relevant info necessary of the start and end rooms
                          #graph is the graph that the dijkstras algorithm acts on
    lowest_dist = float('inf') #initialise the lowest distance (must be inf or smth large such the next distance will always be less)
    distsandpath = {} #initialise the distances and path dictionary, to map each path and its distance
    for node_1 in info[0][2]: #for each node the start corridor or room is connected to
        for node_2 in info[1][2]: #for each node end corridor or room is connected to
            distance,path = dijkstra_recursive(graph,node_1,node_2) #find distance and path from dijkstras algorithm between the two nodes
            distsandpath[f"{node_1}:{node_2}"] = [(total_path_dist(distance,path)+CorridorDists[info[0][1]]+CorridorDists[info[1][1]]),path]
            #add the corresponding distance and path tothe distsandpath dictionary
        #as a result a dictionary of all possible paths from start room to end room are obtained
        #as each room is connected to two nodes, then between the start and end room there are 2 start and 2 end nodes
        #so a totla of 4 possible paths can be taken, hence a dictioanry is used to map each path and its total path distance
    for path in distsandpath: #for each path in distandpath
        if distsandpath[path][0] < lowest_dist: #if the distance of the path is less than the lowest distance,
            lowest_dist = distsandpath[path][0] #then set this as the new lowest distance

    return [(path,distsandpath[path]) for path in distsandpath if distsandpath[path][0]==lowest_dist]
    #return a pair of path and the path distance if the path is the lowest distance path
    #Basically this filters to get the lowest distance path
```

```
137
138 def decodepathofnodestocorridors(path,start,end): #A function to onvert the node path into a path of corridors
139     Cpath = [start] #add the start corridor to the corridor path
140     for node in range(len(path)-1): #loop through each node in path
141         Cpath.append(NodeConnections[path[node]][path[node+1]]) #add to corridor path the corridor connecting the current node and next ndoe
142     Cpath.append(end) #add the end corridor to the corridor path
143     return Cpath #return the corridor path
144
145 def findfloor(corridor): #function to find the floor that a corridor lies within
146     for floor in CorridorFloorMapper: #for each floor in floor mapping list
147         for current_corridor in CorridorFloorMapper[floor]: #for each corridor in corridor mapping list of a certain floor
148             if corridor == current_corridor: #if the current corridor is the corridor of interest
149                 return floor #return the floor that the corridor is on
150
151 def maincode(startroom,endroom): #the maincode that starts the pathfinding algorithm, room1 is start room, room2 is end room
152     mapper =list(CorridorContent.items())
153     info = [] #room, corridor, node, room with spacing id , floor (the order that values are stored into info)
154     floor = [] #initialise the floor list
155     info.append(findroomtocorridor(startroom,mapper)) #add relevant information of the starting room to info list
156     info.append(findroomtocorridor(endroom,mapper)) ##add relevant information of the ending room to info list
157     for item in info: #for each item in info (only 2, one for start room and one for end room)
158         floor.append(item[4])#add the floor that they lie on into the floor list
159     if info[0][1] == info[1][1]: #if the corridor of the start room and corridor of end room are the same
160         print("In same corridor") #then no need to navigate as they are in same corridor
161     else:
162         graph = {node:{neighbour_node:CorridorDists[NodeConnections[node][neighbour_node]] for neighbour_node in NodeConnections[node]} for node in NodeConnections}
163         #A list comprehension that converts the NodeConnections' corridors into distances, such it can be operated on by dijkstras algorithm
164         mindist = min_dist(info,graph)#find the minimum distance path from the start room to end room
165         return (mindist[0][1][1],decodepathofnodestocorridors(mindist[0][1][1],info[0][1],info[1][1]),info[0][2],info[1][2],info[0][3],info[1][3],floor)
166         #return the path of nodes, path of corridors,
167         #  the startroom, the endroom, (both without prefixing)
168         #  the startroom, the endroom, (both with prefixing)
169         # the floor list (the floors involved in navigation)
170
```